

Reviewer Pack — Real Radical Rank and SOS Degree for Max-CUT

Entry 937359348344 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 23:34:57 UTC

Real Radical Rank and SOS Degree for Max-CUT

Entry ID: 937359348344 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-07 23:34:57 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Degree of Max-CUT

Statement:

For random Max-CUT instances on n variables, the real radical rank of the moment matrix M_d is $\Theta(\sqrt{n})$ when $d \geq \Omega(\sqrt{n})$. If any instance has real radical rank $o(\sqrt{n})$ for $d = \Theta(\sqrt{n})$, then the SOS degree required to achieve 0.878-approximation is $\omega(\sqrt{n})$.

Rationale (proposer’s reasoning):

Real radical rank captures the minimal number of constraints needed to certify non-negativity, which directly relates to SOS degree. Lower rank implies fewer constraints, requiring higher degree to maintain approximation guarantees. This bridges algebraic geometry’s structure with SOS hierarchy complexity.

Taxonomy category: SOS_DEGREE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4722179f2f4c80b2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_random_3cnf(n, m):
    clauses = []
    for _ in range(m):
        literals = [random.choice([1, -1]) * (i + 1) for i in range(3)]
        random.shuffle(literals)
        clause = f"({' '.join(map(str, literals))}) | "
        clauses.append(clause)
    return ''.join(clauses[:-2])

def qr_decomposition(A):
    n = len(A)
    Q = [[0] * n for _ in range(n)]
    R = [[A[i][j] if i <= j else 0 for j in range(n)] for i in range(n)]

    for k in range(n):
        norm = sum(R[k][i]**2 for i in range(k, n))**0.5
        Q[k][k] = R[k][k] / norm
        for j in range(k + 1, n):
            R[k][j] /= norm
            Q[j][k] = R[k][j]
            for i in range(k, n):
                R[i][j] -= Q[k][i] * R[i][k]

    return Q, R

def real_radical_rank(A):
    _, R = qr_decomposition(A)
    rank = sum(1 for row in R if any(x != 0 for x in row))
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = 3 * n
    cnf_instance = generate_random_3cnf(n, m)

    d = math.isqrt(n)
    A = [[0] * (n + 1) for _ in range(n + 1)]
```

```

for i in range(n):
    A[i][i] = 1
    A[n][i] = -1

M_d = [row[:] + [sum(row[i] * row[j] for i in range(n)) if j == n else 0 for j in range(n + 1)]

rank = real_radical_rank(M_d)
threshold = 0.7 * math.isqrt(n)

conjecture_holds = rank >= threshold
counterexample = "" if conjecture_holds else "real radical rank < 0.7√n"

return {
    "metric_name": "Real Radical Rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = (sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))**0.5
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        counterexample = next(res["counterexample"] for res in results if not res["conjecture_holds"])
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_90a52e41.py", line 82, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_90a52e41.py", line 62, in run_trial

```


Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/937359348344.tar.gz (if generated)